



Events. Experiences. Together.

INTERNAL — ENGINEERING SPECIFICATION

Zordr Events Platform

Complete API Contract

Organizer Portal • Customer App • Admin Portal

Version 2.0 (supersedes v1.0 skeleton)

Prepared for: Zordr Food Private Limited

Date: August 31, 2026

Consistent with: Zordr Master Platform Architecture Document v1.0

1. Introduction

This is the complete API contract for the Zordr Events Platform — covering every endpoint needed to build the Organizer Portal, the Customer (Attendee) App, and the Admin Portal as one coherent backend, consistent with the platform's Master Architecture Document (single source of truth: one NestJS API, one PostgreSQL schema, shared by all three frontends via role-scoped guards).

Each endpoint entry specifies its method, path, purpose, required auth/actor type, request body, response shape, validation rules, permissions, and standard error codes. This supersedes the earlier placeholder "Zordr Ticketing Platform API Contract v1.0" skeleton — every endpoint below is filled in with real request/response fields and business rules drawn from the Organizer, Customer, and Admin module specifications.

1.1 Global Conventions

- Base path: all endpoints are versioned under `/api/v1`.
- Dates/times: ISO 8601, UTC (e.g. 2026-10-12T16:00:00Z); clients convert to local display time.
- Currency: all monetary values are integers in paise (₹1 = 100 paise) to avoid floating-point rounding errors.
- Identifiers: UUID v4 for all resource IDs; events additionally expose a human-readable slug for public URLs.
- Deletes: soft delete by default (records retained, flagged deleted, excluded from normal queries) — required for financial/audit retention.
- Pagination: page (default 1) and limit (default 20, max 100) query params; responses include a pagination { page, limit, total, totalPages } object.
- Filtering/sorting: list endpoints accept relevant filter query params (see each endpoint) and, where noted, a sort param (e.g. sort=-createdAt).
- Response envelope — success: { "success": true, "data": { ... } }
- Response envelope — error: { "success": false, "error": { "code": "STRING_CODE", "message": "Human-readable", "details": {} } }
- Idempotency: mutating endpoints that are financially relevant (order/payment creation, settlement generation, refunds, webhook processing) accept an Idempotency-Key header and are safe to retry.
- Auth header: Authorization: Bearer <accessToken> on every authenticated request; the token's actor-type claim (organizer / customer / admin_employee) plus role/permission drives authorization — see Master Architecture Doc §8.

1.2 Standard Error Codes

Code	Name	Typical Cause
400	Bad Request	Malformed request, missing required field format
401	Unauthorized	Missing, invalid, or expired access token
403	Forbidden	Valid token but actor lacks permission for this action
404	Not Found	Resource does not exist or is not visible to this actor
409	Conflict	State conflict — e.g. duplicate submission, already processed, quantity below sold
422	Unprocessable Entity	Field-level validation failure (business rule, not just format)
429	Too Many Requests	Rate limit exceeded (OTP, login, checkout endpoints)
500	Internal Server Error	Unexpected server-side failure

1.3 Actor Types & Auth Summary

Actor	Login Mechanism	Scope
organizer	Mobile OTP	Organizer Portal endpoints; scoped to their own Organizer/Events/Orders/Settlements

Actor	Login Mechanism	Scope
customer	Email/password or Google/Apple SSO	Customer App endpoints; scoped to their own User/Orders/Tickets/Profile
admin_employee	Email/password + optional MFA	Admin Portal endpoints; scoped by Role/RolePermission matrix (see §11)

2. Authentication & Session

Shared Auth module serving all three actor types with a common JWT shape (access + refresh), differentiated by actor-type claim.

POST /api/v1/auth/organizer/otp/send

Purpose: Send a 6-digit OTP to an organizer's mobile number to begin login.

Auth: Public

Request Body: mobile (string, required, 10 digits, E.164 with +91)

Response: requestId (uuid), expiresInSeconds (integer)

Validation: Mobile must be exactly 10 digits; rate-limited to 5 requests per number per hour.

Permissions: Public — no token required.

Errors: 400 Invalid mobile, 429 Too Many Requests, 500

POST /api/v1/auth/organizer/otp/verify

Purpose: Verify OTP and issue organizer session tokens.

Auth: Public

Request Body: requestId (uuid, required), otp (string, required, 6 digits)

Response: accessToken, refreshToken, expiresIn, organizer { id, orgName, kycStatus }

Validation: OTP must match and not be expired (TTL 5–10 min); max 5 attempts before logout.

Permissions: Public.

Errors: 400 Invalid/expired OTP, 401, 429 Locked out, 500

Notes: If organizer has no Organizer record yet, response includes onboardingRequired: true.

POST /api/v1/auth/customer/signup

Purpose: Create a new attendee account with email/password.

Auth: Public

Request Body: name, email, password, phone (all required)

Response: accessToken, refreshToken, user { id, name, email }

Validation: Email unique and valid format; password min 8 chars with complexity rule; phone 10 digits.

Permissions: Public.

Errors: 400, 409 Email already registered, 422, 500

POST /api/v1/auth/customer/login

Purpose: Attendee email/password login.

Auth: Public

Request Body: email (required), password (required)

Response: accessToken, refreshToken, user { id, name, email }

Validation: Credentials must match a hashed record via bcrypt/argon2 comparison.

Permissions: Public.

Errors: 400, 401 Invalid credentials, 429 Rate limited, 500

POST /api/v1/auth/customer/oauth/{provider}

Purpose: Complete Google or Apple SSO login/signup for an attendee.

Auth: Public

Path Params: provider (enum: google, apple)

Request Body: idToken (string, required — provider-issued token)

Response: accessToken, refreshToken, user { id, name, email }

Validation: Provider token must be verified server-side against Google/Apple; account is linked to an existing User by verified email if one exists.

Permissions: Public.

Errors: 400 Invalid token, 401, 500

POST /api/v1/auth/admin/login

Purpose: Internal employee email/password login, with optional MFA challenge.

Auth: Public (internal network only)

Request Body: email (required), password (required), mfaCode (string, optional)

Response: accessToken, refreshToken, employee { id, name, role }, mfaRequired (boolean)

Validation: Account must be Active; MFA code validated via TOTP if enabled for the account.

Permissions: Restricted to Zordr's internal network (VPN/IP allowlist) at the infrastructure layer.

Errors: 400, 401, 403 Inactive account, 429 Locked out, 500

Notes: Every successful login writes an AuditLog entry (actor, IP, device, timestamp).

POST /api/v1/auth/refresh

Purpose: Exchange a valid refresh token for a new access token (shared across all three actor types).

Auth: Refresh token (body)

Request Body: refreshToken (required)

Response: accessToken, expiresIn

Validation: Refresh token must be unexpired and not revoked.

Permissions: N/A — token-based.

Errors: 401 Invalid/expired refresh token, 500

POST /api/v1/auth/logout

Purpose: Revoke the current session's refresh token.

Auth: Bearer (any actor)

Response: success: true

Permissions: Any authenticated actor.

Errors: 401, 500

POST /api/v1/auth/customer/forgot-password

Purpose: Trigger a password-reset email for an attendee account.

Auth: Public

Request Body: email (required)

Response: success: true (always, to avoid email enumeration)

Permissions: Public.

Errors: 429 Rate limited, 500

POST /api/v1/auth/customer/reset-password

Purpose: Complete a password reset using the emailed token.

Auth: Public

Request Body: resetToken (required), newPassword (required)

Response: success: true

Validation: Token unexpired and unused; password meets complexity rule.

Permissions: Public.

Errors: 400 Invalid/expired token, 422, 500

3. Organizer Onboarding & Profile

KYC-style onboarding wizard (5 steps) and post-approval profile/bank/document management. See Organizer Portal Doc §4.2.

POST /api/v1/organizer/onboarding

Purpose: Create or update the organizer's onboarding draft (org info, contact, payout, documents) — called incrementally per wizard step.

Auth: Bearer (Organizer)

Request Body: step (enum: org_details|contact_address|payout|documents), payload (object — shape depends on step; see Organizer Portal Doc §4.2)

Response: onboarding { id, currentStep, completedSteps[], status }

Validation: IFSC must be 11 chars and pass registry validation; account number must equal confirmAccountNumber; logo/doc files restricted to PNG/JPG/PDF, max size per type.

Permissions: Organizer Owner only.

Errors: 400, 401, 422 Field-level validation, 500

POST /api/v1/organizer/onboarding/submit

Purpose: Submit the completed onboarding application for KYC review.

Auth: Bearer (Organizer)

Response: onboarding { status: 'submitted', submittedAt }

Validation: All required fields across all steps must be present.

Permissions: Organizer Owner only.

Errors: 400 Incomplete application, 401, 409 Already submitted, 500

Notes: Triggers an email + WhatsApp notification and creates a review item in the Admin Organizers queue.

GET /api/v1/organizer/onboarding

Purpose: Fetch the current onboarding draft/status.

Auth: Bearer (Organizer)

Response: onboarding { ...full draft, status, rejectionReason? }

Permissions: Organizer Owner only.

Errors: 401, 404, 500

GET /api/v1/organizer/profile

Purpose: Fetch the organizer's full profile (org info, contact, branding).

Auth: Bearer (Organizer)

Response: organizer { id, orgName, orgType, logoUrl, contact {...}, kycStatus }

Permissions: Organizer Owner or Team Member (read).

Errors: 401, 404, 500

PUT /api/v1/organizer/profile

Purpose: Update organizer profile/contact/branding fields post-onboarding.

Auth: Bearer (Organizer)

Request Body: orgName?, logoUrl?, contact { email?, phone?, website?, instagram?, address?, city?, state?, pincode? }

Response: organizer { ...updated }

Validation: Same field-level rules as onboarding.

Permissions: Organizer Owner only.

Errors: 400, 401, 403, 500

PUT /api/v1/organizer/bank

Purpose: Update payout bank/UPI details.

Auth: Bearer (Organizer)

Request Body: accountHolder, accountNumber, confirmAccountNumber, ifsc, upiId?

Response: payoutAccount { id, bankName, branch, maskedAccountNumber, verificationStatus }

Validation: Account numbers must match; IFSC validated and used to auto-fill bank/branch; triggers penny-drop verification.

Permissions: Organizer Owner only.

Errors: 400, 401, 409 Verification failed, 500

Notes: Change is audit-logged given financial sensitivity.

PUT /api/v1/organizer/documents

Purpose: Upload or replace KYC documents (PAN, GST, org registration, other).

Auth: Bearer (Organizer)

Request Body: docType (enum: pan|gst|registration|other), fileUrl (from pre-signed upload)

Response: document { id, docType, status: 'pending_review' }

Validation: File type/size restrictions per docType.

Permissions: Organizer Owner only.

Errors: 400, 401, 422, 500

POST /api/v1/organizer/documents/upload-url

Purpose: Request a pre-signed S3 URL to upload a document or logo directly from the browser.

Auth: Bearer (Organizer)

Request Body: fileName, contentType, purpose (enum: logo|pan|gst|registration|other)

Response: uploadUrl, fileUrl, expiresIn

Permissions: Organizer Owner only.

Errors: 400, 401, 500

4. Organizer Dashboard

GET /api/v1/organizer/dashboard

Purpose: Fetch aggregated KPI cards, recent activity, and upcoming events for the organizer home screen.

Auth: Bearer (Organizer)

Response: kpis { activeEvents, ticketsSold, totalRevenue, todayCheckins, pendingSettlement, deltas{} }, recentActivity[], upcomingEvents[]

Permissions: Organizer Owner or Team Member with Dashboard view.

Errors: 401, 500

Notes: Backed by a cached aggregation refreshed every 1–5 min or on relevant events (sale, check-in, settlement).

5. Event Management (Organizer)

Backs the 7-step Create Event wizard plus the My Events list and lifecycle actions. Publishing here creates a submission for Admin review — it does not directly make an event public (see Master Architecture Doc §3).

GET /api/v1/organizer/events

Purpose: List the organizer's events with status/date/category filters.

Auth: Bearer (Organizer)**Query Params:** status?, category?, dateFrom?, dateTo?, search?, page=1, limit=20**Response:** events[] { id, name, status, startAt, venue, ticketsSold, ticketsTotal, revenue }, pagination { page, limit, total }**Permissions:** Organizer Owner or Team Member with Events view.**Errors:** 401, 500

POST /api/v1/organizer/events

Purpose: Create a new event in Draft status (Step 1 — Basic Info).**Auth:** Bearer (Organizer)**Request Body:** name, category, shortDescription (≤300 chars), longDescription? (≤2000 chars)**Response:** event { id, status: 'draft', ...fields }**Validation:** name, category, shortDescription required.**Permissions:** Organizer Owner or Team Member with Events create.**Errors:** 400, 401, 422, 500

GET /api/v1/organizer/events/{id}

Purpose: Fetch full event detail for editing in the wizard.**Auth:** Bearer (Organizer)**Path Params:** id (uuid)**Response:** event { ...all fields across all 7 steps }**Permissions:** Owning organizer only.**Errors:** 401, 403, 404, 500

PATCH /api/v1/organizer/events/{id}

Purpose: Update Basic Info fields on a draft or published event.**Auth:** Bearer (Organizer)**Path Params:** id (uuid)**Request Body:** name?, category?, shortDescription?, longDescription?**Response:** event { ...updated }**Validation:** Once an event has ≥1 paid order, materially destructive changes are blocked or require an attendee-notification confirmation.**Permissions:** Owning organizer only.**Errors:** 400, 401, 403, 409 Locked field, 422, 500

DELETE /api/v1/organizer/events/{id}

Purpose: Delete a draft event (soft delete).**Auth:** Bearer (Organizer)**Path Params:** id (uuid)**Response:** success: true**Validation:** Only permitted while status = draft and no orders exist.**Permissions:** Organizer Owner only.**Errors:** 401, 403, 409 Cannot delete non-draft, 500

PUT /api/v1/organizer/events/{id}/venue

Purpose: Set venue details (Step 2).**Auth:** Bearer (Organizer)**Path Params:** id (uuid)**Request Body:** mode (enum: physical|online|hybrid), venueName?, address? (≤300 chars), lat?, lng?, instructions? (≤300 chars), facilities[]?, meetingLink? (required if online/hybrid)

Response: event.venue { ...updated }

Validation: address/lat/lng required if mode is physical or hybrid; meetingLink required if online or hybrid.

Permissions: Owning organizer only.

Errors: 400, 401, 403, 422, 500

PUT /api/v1/organizer/events/{id}/schedule

Purpose: Set the main date/time and optional multi-session schedule (Step 3).

Auth: Bearer (Organizer)

Path Params: id (uuid)

Request Body: startAt, endAt, sessions?[] { title, description?, startTime, endTime }

Response: event { startAt, endAt, sessions[] }

Validation: endAt > startAt; each session's time range must fall within [startAt, endAt] and endTime > startTime.

Permissions: Owning organizer only.

Errors: 400, 401, 403, 422, 500

POST /api/v1/organizer/events/{id}/media

Purpose: Attach a banner, gallery image, or promo video (Step 4).

Auth: Bearer (Organizer)

Path Params: id (uuid)

Request Body: type (enum: banner|gallery|video), url (from pre-signed upload or YouTube URL), sortOrder?

Response: mediaAsset { id, type, url, sortOrder }

Validation: Banner required before publish; gallery max 10 images; file size/dimension guidance enforced client-side and re-checked server-side.

Permissions: Owning organizer only.

Errors: 400, 401, 403, 422, 500

DELETE /api/v1/organizer/events/{id}/media/{mediaId}

Purpose: Remove a gallery image or the promo video.

Auth: Bearer (Organizer)

Path Params: id (uuid), mediaId (uuid)

Response: success: true

Permissions: Owning organizer only.

Errors: 401, 403, 404, 500

POST /api/v1/organizer/events/{id}/publish

Purpose: Submit the event for admin review, or schedule/keep as draft (Step 7).

Auth: Bearer (Organizer)

Path Params: id (uuid)

Request Body: visibility (enum: publish_now|schedule|draft), scheduledAt? (required if schedule)

Response: event { status: 'submitted', submittedAt }

Validation: Pre-publish checklist must be fully satisfied (name/category/description, venue, schedule, ≥1 visible ticket type, banner present). publish_now/schedule transitions status to submitted, not published directly.

Permissions: Organizer Owner only.

Errors: 400 Checklist incomplete, 401, 403, 422, 500

Notes: Creates an EventReview record and enters the Admin moderation queue — see Master Architecture Doc §3. status only becomes 'published' after Admin approval.

POST /api/v1/organizer/events/{id}/pause

Purpose: Temporarily pause ticket sales on a published event without cancelling it.

Auth: Bearer (Organizer)
Path Params: id (uuid)
Response: event { salesPaused: true }
Permissions: Owning organizer only.
Errors: 401, 403, 409, 500

POST /api/v1/organizer/events/{id}/duplicate

Purpose: Create a new draft event pre-filled from an existing one.
Auth: Bearer (Organizer)
Path Params: id (uuid)
Response: event { id (new), status: 'draft' }
Permissions: Owning organizer only.
Errors: 401, 403, 404, 500
Notes: Ticket sold counts, orders, and check-ins are never copied.

POST /api/v1/organizer/events/{id}/cancel

Purpose: Cancel a published event (Danger Zone).
Auth: Bearer (Organizer)
Path Params: id (uuid)
Request Body: confirmationText (must equal event name), reason
Response: event { status: 'cancelled' }
Validation: Secondary confirmation required (type-to-confirm).
Permissions: Organizer Owner only.
Errors: 400 Confirmation mismatch, 401, 403, 500
Notes: Triggers automated attendee refund workflow and bulk notification.

6. Ticket Types

GET /api/v1/organizer/events/{id}/tickets

Purpose: List ticket types configured for an event.
Auth: Bearer (Organizer)
Path Params: id (uuid)
Response: ticketTypes[] { id, name, price, quantity, sold, available, visible, saleStart, saleEnd, purchaseLimit }
Permissions: Owning organizer only.
Errors: 401, 403, 500

POST /api/v1/organizer/events/{id}/tickets

Purpose: Create a new ticket type (Step 5).
Auth: Bearer (Organizer)
Path Params: id (uuid)
Request Body: name, description?, price (paise, ≥0), quantity (integer, ≥1), saleStart?, saleEnd?, visible=true, purchaseLimit?
Response: ticketType { id, ...fields, sold: 0, available: quantity }
Validation: price and quantity required and non-negative; saleEnd > saleStart if both given.
Permissions: Owning organizer only.
Errors: 400, 401, 403, 422, 500

PUT /api/v1/organizer/tickets/{id}

Purpose: Update a ticket type's price, quantity, visibility, or limits.
Auth: Bearer (Organizer)

Path Params: id (uuid)

Request Body: name?, price?, quantity?, visible?, saleStart?, saleEnd?, purchaseLimit?

Response: ticketType { ...updated }

Validation: quantity can only be decreased down to the current sold count, never below; sold/available are never client-settable.

Permissions: Owning organizer only.

Errors: 400, 401, 403, 409 Quantity below sold, 422, 500

DELETE /api/v1/organizer/tickets/{id}

Purpose: Delete a ticket type.

Auth: Bearer (Organizer)

Path Params: id (uuid)

Response: success: true

Validation: Blocked (or requires attendee-notification flow) if sold > 0.

Permissions: Owning organizer only.

Errors: 401, 403, 409 Has sales, 500

7. Registration Form Builder

GET /api/v1/organizer/events/{id}/registration-form

Purpose: Fetch the registration form configuration and settings (Step 6).

Auth: Bearer (Organizer)

Path Params: id (uuid)

Response: registration { enabled, deadline, maxAttendees, waitlistEnabled, fields[] { id, label, fieldType, required, sortOrder }, consentText, consentMandatory }

Permissions: Owning organizer only.

Errors: 401, 403, 500

PUT /api/v1/organizer/events/{id}/registration-form

Purpose: Configure the registration form: fields, deadline, capacity, waitlist, consent text.

Auth: Bearer (Organizer)

Path Params: id (uuid)

Request Body: enabled, deadline?, maxAttendees?, waitlistEnabled, fields[] { label, fieldType (enum: text|email|phone|dropdown|...), required, sortOrder, options?[] }, consentText? (≤300 chars), consentMandatory

Response: registration { ...updated }

Validation: Field definitions are versioned — editing after attendees have registered does not invalidate prior Registration responses.

Permissions: Owning organizer only.

Errors: 400, 401, 403, 422, 500

8. Scanner / Check-in

Used by the Organizer Portal's Scanner screen; also callable by a Scanner Operator role scoped to specific events.

GET /api/v1/organizer/events/{id}/checkin/stats

Purpose: Fetch live check-in stats for the Scanner screen.

Auth: Bearer (Organizer / Scanner Operator)

Path Params: id (uuid)

Response: stats { totalRegistrations, checkedIn, pending, invalidScans }

Permissions: Owning organizer, Team Member, or Scanner Operator scoped to this event.

Errors: 401, 403, 500

POST /api/v1/organizer/events/{id}/checkin/scan

Purpose: Validate a scanned QR payload and mark the ticket checked in.

Auth: Bearer (Organizer / Scanner Operator)

Path Params: id (uuid)

Request Body: qrPayload (string, signed)

Response: result (enum: valid|already_checked_in|invalid|wrong_event), ticket? { id, ticketType, holderName }

Validation: HMAC signature must verify; duplicate scans rejected as already_checked_in unless allowMultipleEntries is enabled for the event.

Permissions: Owning organizer, Team Member, or Scanner Operator scoped to this event.

Errors: 400 Invalid payload, 401, 403, 404, 500

Notes: Every scan attempt (valid or invalid) is logged to CheckIn. Client should queue scans locally and sync on reconnect if offline.

POST /api/v1/organizer/events/{id}/checkin/manual

Purpose: Manually check in an attendee by name or order ID search (fallback for poor connectivity/damaged QR).

Auth: Bearer (Organizer / Scanner Operator)

Path Params: id (uuid)

Request Body: query (string — name, email, or order ID), ticketId (uuid, to confirm selection)

Response: result (enum: valid|already_checked_in), ticket { ...details }

Permissions: Owning organizer, Team Member, or Scanner Operator scoped to this event.

Errors: 400, 401, 403, 404, 500

POST /api/v1/organizer/events/{id}/checkin/{checkinId}/undo

Purpose: Reverse an accidental check-in.

Auth: Bearer (Organizer)

Path Params: id (uuid), checkinId (uuid)

Response: success: true, ticket { status: 'valid' }

Permissions: Owning organizer only (not Scanner Operator).

Errors: 401, 403, 404, 500

Notes: Audit-logged.

9. Settlements (Organizer View)

GET /api/v1/organizer/settlements

Purpose: List settlements across the organizer's events.

Auth: Bearer (Organizer)

Query Params: eventId?, status?, dateFrom?, dateTo?, page=1, limit=20

Response: settlements[] { id, eventId, period, grossSales, platformFee, gatewayFee, refunds, netPayout, status, transferDate }

Permissions: Organizer Owner or Team Member with Settlements view.

Errors: 401, 500

GET /api/v1/organizer/settlements/{id}

Purpose: Fetch a single settlement's full detail including its transaction ledger.

Auth: Bearer (Organizer)

Path Params: id (uuid)

Response: settlement { ...summary, transactions[] { orderId, buyerName, ticketType, amount, paymentMethod, status, timestamp } }

Permissions: Owning organizer only.

Errors: 401, 403, 404, 500

GET /api/v1/organizer/settlements/{id}/invoice**Purpose:** Download the settlement invoice/statement as PDF.**Auth:** Bearer (Organizer)**Path Params:** id (uuid)**Response:** Binary PDF stream (application/pdf)**Permissions:** Owning organizer only.**Errors:** 401, 403, 404, 500

10. Organizer Team & Event Settings

GET /api/v1/organizer/team**Purpose:** List invited team members and their scoped roles.**Auth:** Bearer (Organizer)**Response:** members[] { id, name, email, role, eventScope[], status }**Permissions:** Organizer Owner only.**Errors:** 401, 500**POST /api/v1/organizer/team/invite****Purpose:** Invite a team member with a scoped role (e.g. Event Manager, Scanner Operator, Finance Viewer).**Auth:** Bearer (Organizer)**Request Body:** email, role (enum), eventScope?[] (uuid[] — omit for org-wide)**Response:** invite { id, status: 'pending' }**Validation:** Issues a time-limited, single-use invite token.**Permissions:** Organizer Owner only.**Errors:** 400, 401, 409 Already invited, 500**GET /api/v1/organizer/settings/{eventId}****Purpose:** Fetch the full settings bundle for an event (branding, additional preferences).**Auth:** Bearer (Organizer)**Path Params:** eventId (uuid)**Response:** settings { branding{}, allowWaitlist, showAttendeeList, ageRestriction, showTerms }**Permissions:** Owning organizer only.**Errors:** 401, 403, 404, 500**PATCH /api/v1/organizer/settings/{eventId}****Purpose:** Update event-level settings toggles.**Auth:** Bearer (Organizer)**Path Params:** eventId (uuid)**Request Body:** allowWaitlist?, showAttendeeList?, ageRestriction?, showTerms?**Response:** settings { ...updated }**Permissions:** Owning organizer only.**Errors:** 400, 401, 403, 500

11. Customer Discovery

Public-facing endpoints backing the Home feed and Event Details screen. Only Published events are ever returned — see Master Architecture Doc §9.1 (Organizer Doc) equivalent rule.

GET /api/v1/discover/home

Purpose: Fetch the Home feed: featured events, categories, upcoming, trending near you.

Auth: Public (optional Bearer for personalization)

Query Params: lat?, lng?

Response: featured[], categories[], upcoming[], trending[]

Permissions: Public; only Published events are ever returned.

Errors: 500

GET /api/v1/discover/search

Purpose: Search/filter events by keyword, category, date, or city.

Auth: Public

Query Params: q?, category?, city?, dateFrom?, dateTo?, page=1, limit=20

Response: events[] { id, slug, name, category, startAt, venue, priceFrom }, pagination{}

Permissions: Public; only Published events indexed.

Errors: 500

GET /api/v1/events/{slug}

Purpose: Fetch full public Event Details payload including live ticket availability.

Auth: Public (optional Bearer for wishlist state)

Path Params: slug (string)

Response: event { name, description, venue, schedule, gallery, ticketTypes[] { name, price, available }, registrationDeadline, faqs, terms }

Permissions: Public; 404 if event is not Published — Draft/Pending Review/Sent-Back events are never resolvable even by direct link.

Errors: 404, 500

Notes: Ticket availability and the registration countdown are computed server-side at request time, never cached beyond a few seconds.

12. Orders & Checkout

Booking flow steps 1-2 and order lifecycle. Ticket inventory is only decremented on payment capture, never at order creation.

POST /api/v1/orders

Purpose: Create a draft order for selected ticket types and quantities (Booking Step 1 → 2 transition).

Auth: Bearer (Customer)

Request Body: eventId, items[] { ticketTypeId, quantity }

Response: order { id, status: 'draft', items[], subtotal }

Validation: Quantity capped by both TicketType.available and purchaseLimit per buyer identity; sale window enforced server-side regardless of client display.

Permissions: Authenticated attendee.

Errors: 400, 401, 404, 409 Sold out / limit exceeded, 422, 500

POST /api/v1/orders/{id}/registration

Purpose: Submit registration form responses for the order (Booking Step 2).

Auth: Bearer (Customer)

Path Params: id (uuid)

Request Body: responses[] { fieldId, value }

Response: order { registrationComplete: true }

Validation: Form is rendered dynamically from the event's RegistrationField definitions; all required fields must be present and pass type-specific validation. Locked after submission.

Permissions: Order owner only.

Errors: 400, 401, 403, 422, 500

GET /api/v1/orders/{id}

Purpose: Fetch order status/detail — used by Order Success and by polling after a payment redirect.

Auth: Bearer (Customer)

Path Params: id (uuid)

Response: order { id, status, amount, tickets[], paymentStatus }

Permissions: Order owner only.

Errors: 401, 403, 404, 500

Notes: The redirect page is only a UX hint; this endpoint reflects the authoritative, webhook-driven status.

GET /api/v1/customers/me/orders

Purpose: List the authenticated attendee's order history.

Auth: Bearer (Customer)

Query Params: page=1, limit=20

Response: orders[] { id, eventName, amount, status, createdAt }, pagination{}

Permissions: Self only.

Errors: 401, 500

POST /api/v1/orders/{id}/cancel

Purpose: Cancel a draft/unpaid order (does not cancel an already-issued ticket — see Refunds).

Auth: Bearer (Customer)

Path Params: id (uuid)

Response: order { status: 'cancelled' }

Validation: Only permitted while status is draft or awaiting_payment.

Permissions: Order owner only.

Errors: 401, 403, 409 Already paid, 500

Notes: Releases any temporary inventory hold on the associated TicketType.

13. Payments

Razorpay-centric payment integration. All state transitions are event-driven off signed webhooks — the client redirect is a UX hint only, never authoritative.

POST /api/v1/payments/create-order

Purpose: Create a Razorpay payment session for a Zodr order (Booking Step 3).

Auth: Bearer (Customer)

Request Body: orderId (uuid)

Response: razorpayOrderId, amount, currency: 'INR', keyId

Validation: Convenience Fee, Platform Fee, and GST are computed server-side from the current FeeConfig and frozen onto the order at this point.

Permissions: Order owner only.

Errors: 400, 401, 403, 409 Order not in payable state, 500

POST /api/v1/payments/verify

Purpose: Client-side callback verification after the Razorpay checkout modal completes (a UX hint only, not authoritative).

Auth: Bearer (Customer)

Request Body: razorpayOrderId, razorpayPaymentId, razorpaySignature

Response: status (enum: pending|verifying) — final confirmation always comes via /payments/webhook

Permissions: Order owner only.

Errors: 400 Signature mismatch, 401, 500

POST /api/v1/payments/webhook

Purpose: Razorpay server-to-server webhook — the single authoritative source for payment capture/failure/refund events.

Auth: Signature-verified (Razorpay webhook secret), no user token

Request Body: Razorpay's standard webhook payload (event, payload.payment.entity, etc.)

Response: 200 OK (empty body)

Validation: Webhook signature must be verified before processing; handler is idempotent (safe to receive the same event twice).

Permissions: Not user-facing — server-to-server only, IP-restricted to Razorpay's published ranges where possible.

Errors: 400 Bad signature, 500

Notes: On payment.captured: creates Ticket + signed QR payload per unit purchased and dispatches confirmation notifications. On payment.failed: releases inventory hold. On refund events: updates Refund/Settlement records.

POST /api/v1/payments/refund

Purpose: Initiate a refund for a captured order (used by Customer Support flow, Organizer cancellation, or Admin refund approval).

Auth: Bearer (Customer, Organizer, or Admin — see Permissions)

Request Body: orderId, amount? (defaults to full amount), reason

Response: refund { id, status: 'processing' }

Validation: Idempotent per orderId; adjusts the affected Settlement's netPayout if that settlement period is still open.

Permissions: Order owner (self-service refund request), Organizer (on event cancellation), or Admin (Operations Manager / Finance Executive role).

Errors: 400, 401, 403, 404, 409 Already refunded, 500

Notes: Refund is processed asynchronously; disclosed SLA is 5–7 business days.

14. Tickets & QR (Customer)

GET /api/v1/customers/me/tickets

Purpose: List the attendee's tickets, filterable by Upcoming/Past/Cancelled.

Auth: Bearer (Customer)

Query Params: bucket? (enum: upcoming|past|cancelled)

Response: tickets[] { id, eventName, venue, startAt, ticketType, quantity, orderId, totalPaid, qrThumbnail }

Permissions: Self only.

Errors: 401, 500

Notes: Bucket is derived server-side from event end datetime and cancellation/refund state, never a client-side date comparison.

GET /api/v1/customers/me/tickets/{ticketId}

Purpose: Fetch full Ticket Details including the QR payload and real-time status.

Auth: Bearer (Customer)

Path Params: ticketId (uuid)

Response: ticket { id, status (enum: valid|checked_in|cancelled|void), qrPayload, ticketType, orderDetails{} }

Permissions: Self only.

Errors: 401, 403, 404, 500

Notes: status reflects the latest CheckIn record — a ticket already scanned at the venue shows checked_in even if reopened here.

15. Customer Profile, Wishlist & Loyalty

GET /api/v1/customers/me/profile

Purpose: Fetch profile info and stats (events attended, interested, loyalty points).

Auth: Bearer (Customer)

Response: profile { name, email, college, branch, year, stats { eventsAttended, eventsInterested, loyaltyPoints } }

Permissions: Self only.

Errors: 401, 500

Notes: eventsAttended is computed from CheckIn records, not ticket purchase, so no-shows are excluded.

PUT /api/v1/customers/me/profile

Purpose: Update personal information and preferences.

Auth: Bearer (Customer)

Request Body: name?, college?, branch?, year?, notificationPreferences?{}

Response: profile { ...updated }

Permissions: Self only.

Errors: 400, 401, 422, 500

POST /api/v1/customers/me/wishlist/{eventId}

Purpose: Add an event to Interests/Wishlist.

Auth: Bearer (Customer)

Path Params: eventId (uuid)

Response: success: true

Permissions: Self only.

Errors: 401, 404, 500

DELETE /api/v1/customers/me/wishlist/{eventId}

Purpose: Remove an event from Interests/Wishlist.

Auth: Bearer (Customer)

Path Params: eventId (uuid)

Response: success: true

Permissions: Self only.

Errors: 401, 404, 500

16. Support (Cross-Platform)

Shared support-ticket submission for organizers and customers; Admin-side queue management is in §20.

POST /api/v1/support/tickets

Purpose: Submit a support request (available to attendees and organizers).

Auth: Bearer (Customer or Organizer)

Request Body: category (enum: tickets|payments|refunds|event_info|orders|accessibility|general), subject, message, orderId? / eventId?

Response: ticket { id, status: 'open' }

Permissions: Any authenticated actor.

Errors: 400, 401, 422, 500

GET /api/v1/support/tickets/{id}

Purpose: Fetch the status/thread of a support ticket the requester raised.

Auth: Bearer (Customer or Organizer)
Path Params: id (uuid)
Response: ticket { id, status, priority, messages[] }
Permissions: Requester only (Admin has separate endpoints — see \$Admin).
Errors: 401, 403, 404, 500

17. Admin — Organizer Governance

Approval authority for Organizer Portal onboarding submissions. See Admin Portal Doc §4.3.

GET /api/v1/admin/organizers

Purpose: List/search/filter all organizers on the platform.
Auth: Bearer (Admin)
Query Params: status?, kycStatus?, city?, search?, page=1, limit=20
Response: organizers[] { id, orgName, kycStatus, activeEvents, totalRevenue, status }, pagination{}
Permissions: Admin role with Organizers:view.
Errors: 401, 403, 500

GET /api/v1/admin/organizers/{id}

Purpose: Fetch full organizer detail: documents, bank/GST, events, revenue, settlements, support tickets, notes.
Auth: Bearer (Admin)
Path Params: id (uuid)
Response: organizer { ...full profile, documents[], events[], settlements[], supportTickets[], notes[] }
Permissions: Admin role with Organizers:view.
Errors: 401, 403, 404, 500

POST /api/v1/admin/organizers/{id}/approve

Purpose: Approve an organizer's onboarding/KYC application.
Auth: Bearer (Admin)
Path Params: id (uuid)
Response: organizer { kycStatus: 'approved' }
Permissions: Admin role with Organizers:edit.
Errors: 401, 403, 404, 409, 500
Notes: Audit-logged. Unblocks the organizer's ability to publish events and receive payouts.

POST /api/v1/admin/organizers/{id}/reject

Purpose: Reject an organizer's onboarding application.
Auth: Bearer (Admin)
Path Params: id (uuid)
Request Body: reason
Response: organizer { kycStatus: 'rejected' }
Permissions: Admin role with Organizers:edit.
Errors: 400, 401, 403, 404, 500

POST /api/v1/admin/organizers/{id}/suspend

Purpose: Suspend an approved organizer.
Auth: Bearer (Admin)
Path Params: id (uuid)
Request Body: reason

Response: organizer { status: 'suspended' }

Permissions: Admin role with Organizers:edit.

Errors: 401, 403, 404, 500

Notes: Cascades: organizer's events are unpublished/hidden from discovery, pending settlements move to on_hold.

POST /api/v1/admin/organizers/{id}/reset-password

Purpose: Force-reset an organizer's login (e.g. security incident).

Auth: Bearer (Admin)

Path Params: id (uuid)

Response: success: true

Permissions: Admin role with Organizers:edit.

Errors: 401, 403, 404, 500

POST /api/v1/admin/organizers/{id}/notes

Purpose: Add an internal note to an organizer's record.

Auth: Bearer (Admin)

Path Params: id (uuid)

Request Body: note (string)

Response: note { id, authorId, note, createdAt }

Permissions: Admin role with Organizers:edit.

Errors: 400, 401, 403, 500

POST /api/v1/admin/organizers/{id}/message

Purpose: Send a direct message/notification to an organizer.

Auth: Bearer (Admin)

Path Params: id (uuid)

Request Body: channel (enum: email|sms|whatsapp), message

Response: success: true

Permissions: Admin role with Organizers:edit.

Errors: 400, 401, 403, 500

18. Admin — Event Moderation

The mandatory publish gate between an organizer's submission and public visibility. See Admin Portal Doc §4.4 and Master Architecture Doc §3.

GET /api/v1/admin/events

Purpose: List all events platform-wide with status filters.

Auth: Bearer (Admin)

Query Params: status?, organizerId?, category?, city?, dateFrom?, dateTo?, search?, page=1, limit=20

Response: events[] { id, name, organizerName, status, startAt, ticketsSold, revenue }, pagination{}

Permissions: Admin role with Events:view.

Errors: 401, 403, 500

GET /api/v1/admin/events/{id}/review

Purpose: Fetch the full Event Review payload: submitted content, checklist state, and review history.

Auth: Bearer (Admin)

Path Params: id (uuid)

Response: event { ...full submitted content }, checklist { bannerImages, details, dateTimeVenue, ticketsPricing, organizerInfo, policiesTerms, contentGuidelines }, reviewHistory[]

Permissions: Admin role with Events:view.

Errors: 401, 403, 404, 500

PUT /api/v1/admin/events/{id}/review/checklist

Purpose: Update individual checklist item states while reviewing.

Auth: Bearer (Admin)

Path Params: id (uuid)

Request Body: item (enum), status (enum: pending|looks_good|flagged), comment?

Response: checklist { ...updated }

Permissions: Admin role with Events:edit.

Errors: 400, 401, 403, 500

POST /api/v1/admin/events/{id}/approve

Purpose: Approve and publish an event, making it visible in Customer discovery.

Auth: Bearer (Admin)

Path Params: id (uuid)

Response: event { status: 'published', publishedAt }

Validation: All checklist items must be looks_good.

Permissions: Admin role with Events:edit.

Errors: 400 Checklist incomplete, 401, 403, 404, 500

Notes: This is the only transition that sets status = published (see Master Architecture Doc §3).

POST /api/v1/admin/events/{id}/send-back

Purpose: Send the event submission back to the organizer for edits.

Auth: Bearer (Admin)

Path Params: id (uuid)

Request Body: notes (string, required)

Response: event { status: 'sent_back' }

Permissions: Admin role with Events:edit.

Errors: 400, 401, 403, 404, 500

Notes: Resubmission after edits re-enters the review queue as a new EventReview cycle; prior cycle history is preserved.

POST /api/v1/admin/events/{id}/reject

Purpose: Reject an event submission outright.

Auth: Bearer (Admin)

Path Params: id (uuid)

Request Body: reason (string, required)

Response: event { status: 'rejected' }

Permissions: Admin role with Events:edit.

Errors: 400, 401, 403, 404, 500

GET /api/v1/admin/events/{id}/preview

Purpose: Render the event exactly as a customer would see it (Preview as Customer).

Auth: Bearer (Admin)

Path Params: id (uuid)

Response: Same shape as GET /events/{slug}, bypassing the Published-only restriction for admin preview.

Permissions: Admin role with Events:view.

Errors: 401, 403, 404, 500

19. Admin — Orders & Customers Oversight

GET /api/v1/admin/orders

Purpose: List every order platform-wide with rich filters.

Auth: Bearer (Admin)

Query Params: eventId?, organizerId?, paymentStatus?, orderStatus?, dateFrom?, dateTo?, search?, page=1, limit=20

Response: orders[] { id, customerName, eventName, organizerName, amount, paymentGateway, paymentStatus, orderStatus, createdAt }, pagination{}

Permissions: Admin role with Orders:view.

Errors: 401, 403, 500

GET /api/v1/admin/orders/export

Purpose: Export the current filtered order list as CSV.

Auth: Bearer (Admin)

Response: Binary CSV stream

Permissions: Admin role with Orders:export.

Errors: 401, 403, 500

POST /api/v1/admin/orders/bulk-action

Purpose: Apply a bulk action (e.g. bulk refund, bulk export) to a set of selected orders.

Auth: Bearer (Admin)

Request Body: orderIds[] (uuid[]), action (enum: refund|export)

Response: job { id, status: 'queued' }

Validation: Processed asynchronously for large selections; fully audit-logged.

Permissions: Admin role with Orders:edit (refund) or Orders:export.

Errors: 400, 401, 403, 500

GET /api/v1/admin/customers

Purpose: List/search/filter all registered attendees.

Auth: Bearer (Admin)

Query Params: status?, city?, search?, dateFrom?, dateTo?, page=1, limit=20

Response: customers[] { id, name, email, city, eventsAttended, lifetimeSpend, lastOrder, status, joinedOn }, pagination{}

Permissions: Admin role with Customers:view.

Errors: 401, 403, 500

POST /api/v1/admin/customers/{id}/block

Purpose: Block a customer from completing future checkouts.

Auth: Bearer (Admin)

Path Params: id (uuid)

Request Body: reason

Response: customer { status: 'blocked' }

Permissions: Admin role with Customers:edit.

Errors: 401, 403, 404, 500

POST /api/v1/admin/customers/{id}/unblock

Purpose: Restore a blocked customer's checkout ability.

Auth: Bearer (Admin)
Path Params: id (uuid)
Response: customer { status: 'active' }
Permissions: Admin role with Customers:edit.
Errors: 401, 403, 404, 500

20. Admin — Settlements, Refunds & Support

Oversight/override layer over the same Settlement/Refund computation used by the Organizer Portal — never a separate copy.

GET /api/v1/admin/settlements

Purpose: List all settlements platform-wide.
Auth: Bearer (Admin)
Query Params: status?, organizerId?, eventId?, dateFrom?, dateTo?, page=1, limit=20
Response: settlements[] { id, organizerName, eventNames[], period, grossSales, platformFee, gatewayFee, refunds, netPayout, status, transferDate }, pagination{}
Permissions: Admin role with Settlements:view.
Errors: 401, 403, 500

POST /api/v1/admin/settlements/generate

Purpose: Trigger settlement generation for a given period (normally run by the scheduled batch job; this is the manual/oversight trigger).
Auth: Bearer (Admin)
Request Body: periodStart, periodEnd, organizerId? (omit for all)
Response: job { id, status: 'queued' }
Permissions: Admin role with Settlements:edit (typically Finance Executive or Super Admin).
Errors: 400, 401, 403, 500
Notes: Idempotent — re-running for an already-generated period does not duplicate settlement rows.

POST /api/v1/admin/settlements/{id}/hold

Purpose: Place a settlement on hold (e.g. pending dispute).
Auth: Bearer (Admin)
Path Params: id (uuid)
Request Body: reason
Response: settlement { status: 'on_hold' }
Permissions: Admin role with Settlements:edit.
Errors: 401, 403, 404, 500

POST /api/v1/admin/settlements/{id}/mark-paid

Purpose: Mark a settlement as paid after payout confirmation from RazorpayX.
Auth: Bearer (Admin)
Path Params: id (uuid)
Request Body: transferReference, transferDate
Response: settlement { status: 'paid' }
Validation: Once marked paid, the settlement becomes immutable; corrections require a linked adjustment record.
Permissions: Admin role with Settlements:edit.
Errors: 401, 403, 404, 409 Already paid, 500

GET /api/v1/admin/settlements/report

Purpose: Download a platform-wide settlement report.

Auth: Bearer (Admin)

Query Params: dateFrom, dateTo, format (enum: csv|pdf)

Response: Binary file stream

Permissions: Admin role with Settlements:export.

Errors: 401, 403, 500

GET /api/v1/admin/refunds

Purpose: List all refund requests platform-wide.

Auth: Bearer (Admin)

Query Params: status?, eventId?, dateFrom?, dateTo?, page=1, limit=20

Response: refunds[] { id, orderId, customerName, eventName, amount, reason, status, requestedOn, processedOn }, pagination{}

Permissions: Admin role with Refunds:view.

Errors: 401, 403, 500

POST /api/v1/admin/refunds/{id}/review

Purpose: Approve or reject a pending refund request.

Auth: Bearer (Admin)

Path Params: id (uuid)

Request Body: decision (enum: approve|reject), notes?

Response: refund { status: 'processed' | 'rejected' }

Permissions: Admin role with Refunds:edit.

Errors: 400, 401, 403, 404, 409, 500

Notes: Approval triggers the same idempotent gateway-refund workflow used elsewhere and adjusts the affected Settlement if still open.

GET /api/v1/admin/support

Purpose: List all support tickets platform-wide (organizer + customer origin).

Auth: Bearer (Admin)

Query Params: status?, category?, priority?, eventId?, search?, page=1, limit=20

Response: tickets[] { id, requesterType, requesterName, subject, category, eventName, status, priority, createdAt }, pagination{}

Permissions: Admin role with Support:view.

Errors: 401, 403, 500

POST /api/v1/admin/support/{id}/reply

Purpose: Reply to / update the status of a support ticket.

Auth: Bearer (Admin)

Path Params: id (uuid)

Request Body: message?, status? (enum: open|pending|resolved)

Response: ticket { ...updated }

Permissions: Admin role with Support:edit.

Errors: 400, 401, 403, 404, 500

Notes: First-response/resolution SLA timers are computed server-side from ticket creation and first-reply timestamps.

21. Admin — Analytics & Reporting

GET /api/v1/admin/analytics/overview

Purpose: Platform-wide KPI summary: GMV, orders, tickets sold, active events, unique customers, conversion rate.

Auth: Bearer (Admin)
Query Params: dateFrom, dateTo
Response: overview { totalGmv, totalOrders, ticketsSold, activeEvents, uniqueCustomers, conversionRate }
Permissions: Admin role with Analytics:view.
Errors: 401, 403, 500
Notes: Served from the reporting/warehouse layer, not live transactional tables.

GET /api/v1/admin/analytics/trends

Purpose: Time-series data for GMV, orders vs customers, and user growth charts.
Auth: Bearer (Admin)
Query Params: metric (enum: gmv|orders_customers|user_growth), granularity (enum: daily|weekly), dateFrom, dateTo
Response: series[] { date, value(s) }
Permissions: Admin role with Analytics:view.
Errors: 401, 403, 500

GET /api/v1/admin/analytics/breakdowns

Purpose: Breakdown data for donuts: revenue by payment method, order status distribution, settlement overview.
Auth: Bearer (Admin)
Query Params: type (enum: payment_method|order_status|settlement_status), dateFrom, dateTo
Response: breakdown[] { label, value, percentage }
Permissions: Admin role with Analytics:view.
Errors: 401, 403, 500

GET /api/v1/admin/analytics/leaderboards

Purpose: Top Events by Revenue, Top Organizers by GMV, Customers by City.
Auth: Bearer (Admin)
Query Params: type (enum: top_events|top_organizers|customers_by_city), dateFrom, dateTo, limit=10
Response: rows[] { rank, label, metric }
Permissions: Admin role with Analytics:view.
Errors: 401, 403, 500

GET /api/v1/admin/analytics/report

Purpose: Download the full analytics report.
Auth: Bearer (Admin)
Query Params: dateFrom, dateTo, format (enum: pdf|csv)
Response: Binary file stream
Permissions: Admin role with Analytics:export.
Errors: 401, 403, 500

22. Admin — Employees & RBAC

The permission matrix here is the literal authorization schema every admin-facing endpoint enforces.

GET /api/v1/admin/employees

Purpose: List internal staff with role/departement/status.
Auth: Bearer (Admin)
Query Params: status?, roleId?, department?, search?, page=1, limit=20
Response: employees[] { id, name, email, role, department, status, lastActiveAt }, pagination{}
Permissions: Admin role with Employees:view.

Errors: 401, 403, 500

POST /api/v1/admin/employees/invite

Purpose: Invite a new internal employee with a role assignment.
Auth: Bearer (Admin)
Request Body: name, email, roleId, department, sendInvitation=true
Response: employee { id, status: 'invited' }
Validation: Issues a time-limited, single-use invite token — never a shared/default password.
Permissions: Admin role with Employees:create.
Errors: 400, 401, 403, 409, 500

PUT /api/v1/admin/employees/{id}

Purpose: Update an employee's role, department, or active status.
Auth: Bearer (Admin)
Path Params: id (uuid)
Request Body: roleId?, department?, status? (enum: active | inactive)
Response: employee { ...updated }
Permissions: Admin role with Employees:edit.
Errors: 400, 401, 403, 404, 500

GET /api/v1/admin/roles

Purpose: List all roles (system + custom) with employee counts.
Auth: Bearer (Admin)
Response: roles[] { id, name, type (enum: system | custom), isDeletable, employeeCount }
Permissions: Admin role with Employees:view.
Errors: 401, 403, 500

POST /api/v1/admin/roles

Purpose: Create a custom role with a full module permission matrix.
Auth: Bearer (Admin)
Request Body: name, description?, permissions[] { module, canView, canCreate, canEdit, canDelete, canExport }
Response: role { id, ...fields }
Permissions: Admin role with Employees:create (typically Super Admin only).
Errors: 400, 401, 403, 422, 500

PUT /api/v1/admin/roles/{id}

Purpose: Update a role's permission matrix.
Auth: Bearer (Admin)
Path Params: id (uuid)
Request Body: permissions[] { module, canView, canCreate, canEdit, canDelete, canExport }
Response: role { ...updated }
Validation: System roles (Super Admin, Event Manager, etc.) cannot be deleted, only have permissions modified.
Permissions: Admin role with Employees:edit.
Errors: 400, 401, 403, 404, 500

DELETE /api/v1/admin/roles/{id}

Purpose: Delete a custom role.
Auth: Bearer (Admin)

Path Params: id (uuid)**Response:** success: true**Validation:** Only permitted for type = custom and isDeletable = true; blocked if employees are still assigned.**Permissions:** Admin role with Employees:delete.**Errors:** 401, 403, 404, 409, 500

23. Admin — Platform Settings & Fee Configuration

Includes the FeeConfig endpoints that resolve the platform-fee reconciliation flagged in the Master Architecture Document §5 (R1) — both the Organizer ticket-pricing disclosure and Settlement computations read from this single source.

GET /api/v1/admin/settings

Purpose: Fetch platform settings bundle (org profile, preferences, plan info).**Auth:** Bearer (Admin)**Response:** settings { organization{}, preferences{ timezone, dateFormat, currency, language, itemsPerPage }, platformInfo{ plan, memberSince, eventsCreated, totalUsers, storageUsed } }**Permissions:** Admin role with Settings:view.**Errors:** 401, 403, 500

PATCH /api/v1/admin/settings

Purpose: Update platform organization details or preferences.**Auth:** Bearer (Admin)**Request Body:** organization?{ name, supportEmail, contactNumber, address, logoUrl }, preferences?{ }**Response:** settings { ...updated }**Permissions:** Admin role with Settings:edit.**Errors:** 400, 401, 403, 500

GET /api/v1/admin/fee-config

Purpose: Fetch the current and historical platform fee configuration.**Auth:** Bearer (Admin)**Response:** feeConfig[] { id, platformFeePct, perTicketFee, gatewayFeePct, effectiveFrom }**Permissions:** Admin role with Settings:view (Finance-scoped).**Errors:** 401, 403, 500

POST /api/v1/admin/fee-config

Purpose: Publish a new platform fee configuration effective from a given date.**Auth:** Bearer (Admin)**Request Body:** platformFeePct, perTicketFee, gatewayFeePct, effectiveFrom**Response:** feeConfig { id, ...fields }**Validation:** Never retroactively alters fees already frozen on existing orders (see Master Architecture Doc §9).**Permissions:** Admin role with Settings:edit (Finance Executive or Super Admin).**Errors:** 400, 401, 403, 422, 500**Notes:** Audit-logged; this is the single source both the Organizer Portal's ticket-pricing disclosure and the Settlements computation read from.

24. Appendix — Full Endpoint Index

Authentication (10 endpoints)

Method	Path
POST	/api/v1/auth/organizer/otp/send
POST	/api/v1/auth/organizer/otp/verify
POST	/api/v1/auth/customer/signup
POST	/api/v1/auth/customer/login
POST	/api/v1/auth/customer/oauth/{provider}
POST	/api/v1/auth/admin/login
POST	/api/v1/auth/refresh
POST	/api/v1/auth/logout
POST	/api/v1/auth/customer/forgot-password
POST	/api/v1/auth/customer/reset-password

Organizer Onboarding (8 endpoints)

Method	Path
POST	/api/v1/organizer/onboarding
POST	/api/v1/organizer/onboarding/submit
GET	/api/v1/organizer/onboarding
GET	/api/v1/organizer/profile
PUT	/api/v1/organizer/profile
PUT	/api/v1/organizer/bank
PUT	/api/v1/organizer/documents
POST	/api/v1/organizer/documents/upload-url

Organizer Dashboard (1 endpoints)

Method	Path
GET	/api/v1/organizer/dashboard

Events (13 endpoints)

Method	Path
GET	/api/v1/organizer/events
POST	/api/v1/organizer/events
GET	/api/v1/organizer/events/{id}
PATCH	/api/v1/organizer/events/{id}
DELETE	/api/v1/organizer/events/{id}
PUT	/api/v1/organizer/events/{id}/venue
PUT	/api/v1/organizer/events/{id}/schedule
POST	/api/v1/organizer/events/{id}/media
DELETE	/api/v1/organizer/events/{id}/media/{mediaId}
POST	/api/v1/organizer/events/{id}/publish

Method	Path
POST	/api/v1/organizer/events/{id}/pause
POST	/api/v1/organizer/events/{id}/duplicate
POST	/api/v1/organizer/events/{id}/cancel

Tickets (4 endpoints)

Method	Path
GET	/api/v1/organizer/events/{id}/tickets
POST	/api/v1/organizer/events/{id}/tickets
PUT	/api/v1/organizer/tickets/{id}
DELETE	/api/v1/organizer/tickets/{id}

Registration Builder (2 endpoints)

Method	Path
GET	/api/v1/organizer/events/{id}/registration-form
PUT	/api/v1/organizer/events/{id}/registration-form

Scanner (4 endpoints)

Method	Path
GET	/api/v1/organizer/events/{id}/checkin/stats
POST	/api/v1/organizer/events/{id}/checkin/scan
POST	/api/v1/organizer/events/{id}/checkin/manual
POST	/api/v1/organizer/events/{id}/checkin/{checkinId}/undo

Settlements (Organizer) (3 endpoints)

Method	Path
GET	/api/v1/organizer/settlements
GET	/api/v1/organizer/settlements/{id}
GET	/api/v1/organizer/settlements/{id}/invoice

Team & Settings (4 endpoints)

Method	Path
GET	/api/v1/organizer/team
POST	/api/v1/organizer/team/invite
GET	/api/v1/organizer/settings/{eventId}
PATCH	/api/v1/organizer/settings/{eventId}

Customer Discovery (3 endpoints)

Method	Path
GET	/api/v1/discover/home
GET	/api/v1/discover/search
GET	/api/v1/events/{slug}

Orders (5 endpoints)

Method	Path
POST	/api/v1/orders
POST	/api/v1/orders/{id}/registration
GET	/api/v1/orders/{id}
GET	/api/v1/customers/me/orders
POST	/api/v1/orders/{id}/cancel

Payments (4 endpoints)

Method	Path
POST	/api/v1/payments/create-order
POST	/api/v1/payments/verify
POST	/api/v1/payments/webhook
POST	/api/v1/payments/refund

Tickets (Customer) (2 endpoints)

Method	Path
GET	/api/v1/customers/me/tickets
GET	/api/v1/customers/me/tickets/{ticketId}

Customer Profile (4 endpoints)

Method	Path
GET	/api/v1/customers/me/profile
PUT	/api/v1/customers/me/profile
POST	/api/v1/customers/me/wishlist/{eventId}
DELETE	/api/v1/customers/me/wishlist/{eventId}

Support (2 endpoints)

Method	Path
POST	/api/v1/support/tickets
GET	/api/v1/support/tickets/{id}

Admin Organizers (8 endpoints)

Method	Path
GET	/api/v1/admin/organizers
GET	/api/v1/admin/organizers/{id}
POST	/api/v1/admin/organizers/{id}/approve
POST	/api/v1/admin/organizers/{id}/reject
POST	/api/v1/admin/organizers/{id}/suspend
POST	/api/v1/admin/organizers/{id}/reset-password
POST	/api/v1/admin/organizers/{id}/notes
POST	/api/v1/admin/organizers/{id}/message

Admin Events (7 endpoints)

Method	Path
GET	/api/v1/admin/events
GET	/api/v1/admin/events/{id}/review
PUT	/api/v1/admin/events/{id}/review/checklist
POST	/api/v1/admin/events/{id}/approve
POST	/api/v1/admin/events/{id}/send-back
POST	/api/v1/admin/events/{id}/reject
GET	/api/v1/admin/events/{id}/preview

Admin Orders/Customers (6 endpoints)

Method	Path
GET	/api/v1/admin/orders
GET	/api/v1/admin/orders/export
POST	/api/v1/admin/orders/bulk-action
GET	/api/v1/admin/customers
POST	/api/v1/admin/customers/{id}/block
POST	/api/v1/admin/customers/{id}/unblock

Admin Settlements/Refunds (7 endpoints)

Method	Path
GET	/api/v1/admin/settlements
POST	/api/v1/admin/settlements/generate
POST	/api/v1/admin/settlements/{id}/hold
POST	/api/v1/admin/settlements/{id}/mark-paid
GET	/api/v1/admin/settlements/report
GET	/api/v1/admin/refunds
POST	/api/v1/admin/refunds/{id}/review

Admin Support (2 endpoints)

Method	Path
GET	/api/v1/admin/support
POST	/api/v1/admin/support/{id}/reply

Admin Analytics (5 endpoints)

Method	Path
GET	/api/v1/admin/analytics/overview
GET	/api/v1/admin/analytics/trends
GET	/api/v1/admin/analytics/breakdowns
GET	/api/v1/admin/analytics/leaderboards
GET	/api/v1/admin/analytics/report

Admin RBAC (7 endpoints)

Method	Path
GET	/api/v1/admin/employees
POST	/api/v1/admin/employees/invite
PUT	/api/v1/admin/employees/{id}
GET	/api/v1/admin/roles
POST	/api/v1/admin/roles
PUT	/api/v1/admin/roles/{id}
DELETE	/api/v1/admin/roles/{id}

Admin Settings (4 endpoints)

Method	Path
GET	/api/v1/admin/settings
PATCH	/api/v1/admin/settings
GET	/api/v1/admin/fee-config
POST	/api/v1/admin/fee-config

Total endpoints in this contract: 115*— End of Document —*